ACADEMY

Section 1 / 25

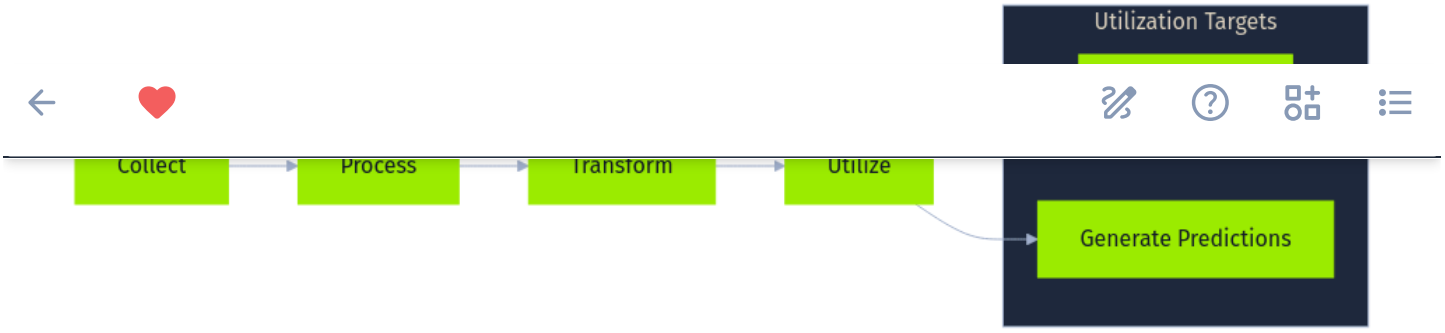
Introduction to AI Data

AI systems learn from data. Performance, reliability, and security all depend on it. When that data is wrong (corrupted, manipulated, or leaked) the model built on it inherits the problem, because it cannot tell the difference between legitimate patterns and patterns an attacker planted.

That single fact drives the entire threat landscape covered in this module.

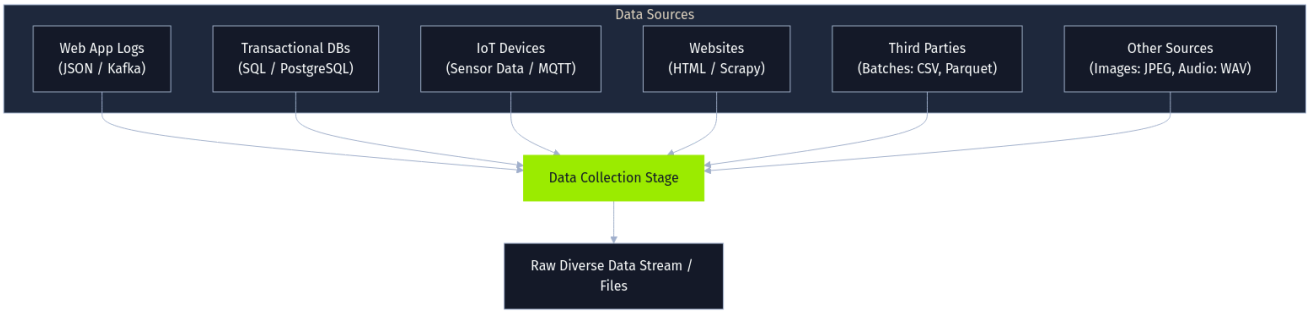
To understand where attacks happen, we need to understand the normal flow first.

The Data Pipeline: Fueling AI Systems



At the heart of most AI implementations lies a `data pipeline`, a sequence of steps designed to collect, process, transform, and ultimately utilize data for tasks such as training models or generating predictions. While the specifics vary greatly depending on the application and organization, a general data pipeline often includes several core stages, frequently leveraging specific technologies and handling diverse data formats.

Data Collection



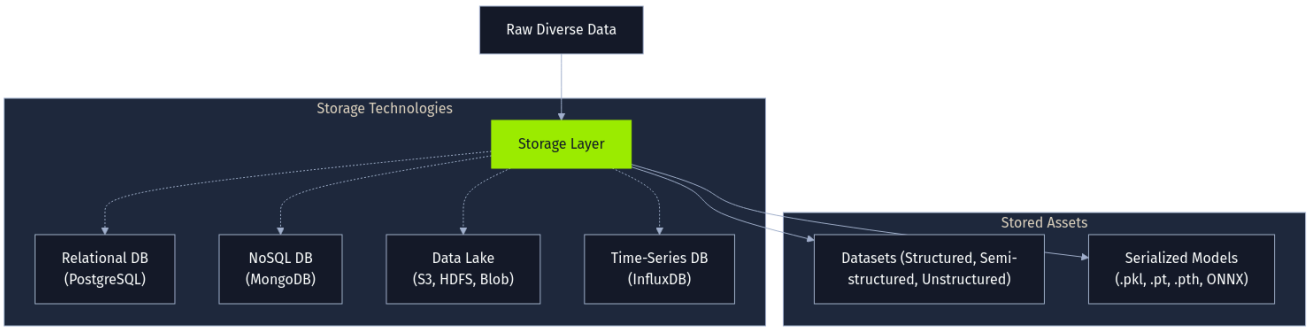
`Data collection` pulls raw information from wherever it originates. User interactions arrive from web applications as `JSON` logs streamed through `Apache Kafka`. Structured transaction records come out of `PostgreSQL`. IoT devices push sensor readings over `MQTT`, while web scrapers extract content from public sites and third parties deliver batch files. The data itself spans everything from `JPEG` images to complex semi-structured formats.

Notice the variety. That is part of the challenge.

Every source is a separate trust boundary, and every format carries different assumptions about integrity. Quality problems introduced here (whether malicious or accidental) ride the pipeline forward into processing, training, and eventually production.

The pipeline does not inherently validate what it collects.

Storage



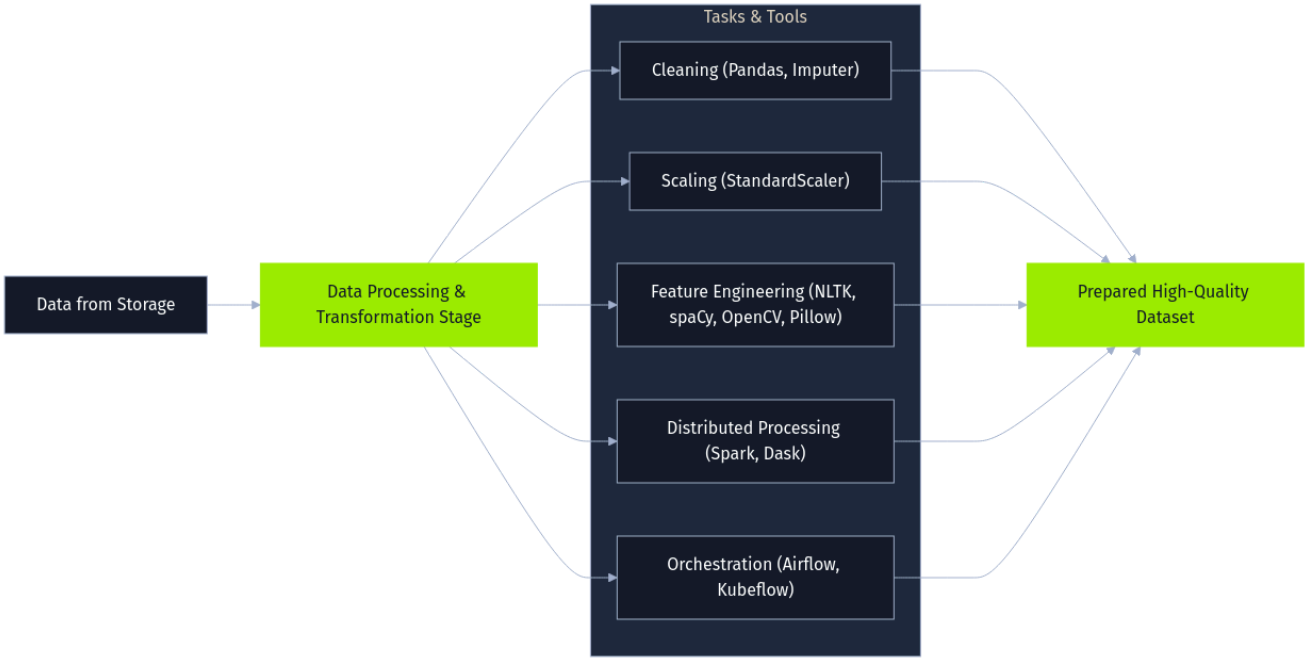
Where the data lands depends entirely on its shape. Structured records go into relational databases like `PostgreSQL`, while semi-structured logs fit better in `NoSQL` stores like `MongoDB`. Large, mixed datasets end up in massive `data lakes` built on distributed file systems or cloud object storage. Time-series data gets specialized tools like `InfluxDB`.

That covers the training data. But there is a second category of stored asset that matters just as much: the models themselves.

Trained models are serialized into files (`.pkl` , `ONNX` , `.pt`) and sit alongside the data in the same storage infrastructure. A `.pkl` file can contain arbitrary Python code, and a `.pt` file can easily be swapped out. If an attacker gets write access to the storage layer, they do not need to poison training data at all. They can replace the model directly.

We will return to this when we cover storage-layer attacks.

Data Processing



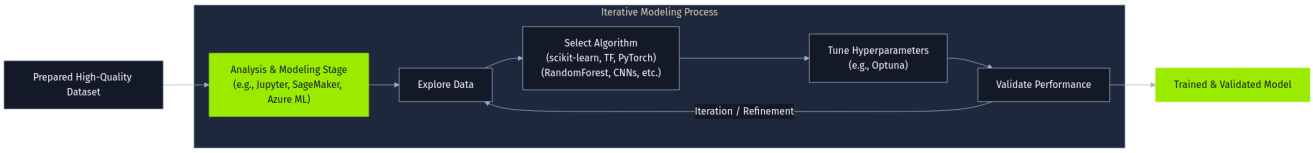
Raw data is almost never usable as-is. `Data processing and transformation` is the stage that turns it into something a model can learn from, and this is exactly where much of the actual engineering effort lives.

Missing values get handled with tools like `Pandas` and `scikit-learn` imputers, while numeric features get scaled to comparable ranges. Then comes `feature engineering` : building new inputs from existing data. That could mean extracting date components from timestamps, generating text embeddings with `spaCy` , or augmenting image datasets with `OpenCV` . At scale, distributed frameworks like `Apache Spark` handle the heavy compute, and orchestrators like `Apache Airflow` keep the workflow in order.

All of this logic is code. Cleaning scripts. Transformation jobs. Feature extractors.

They run automatically, shaping exactly what the model sees. If an attacker compromises the processing logic rather than the raw data itself, the corruption enters the pipeline at a point where nobody is looking for it. The raw data upstream still looks clean.

Modeling



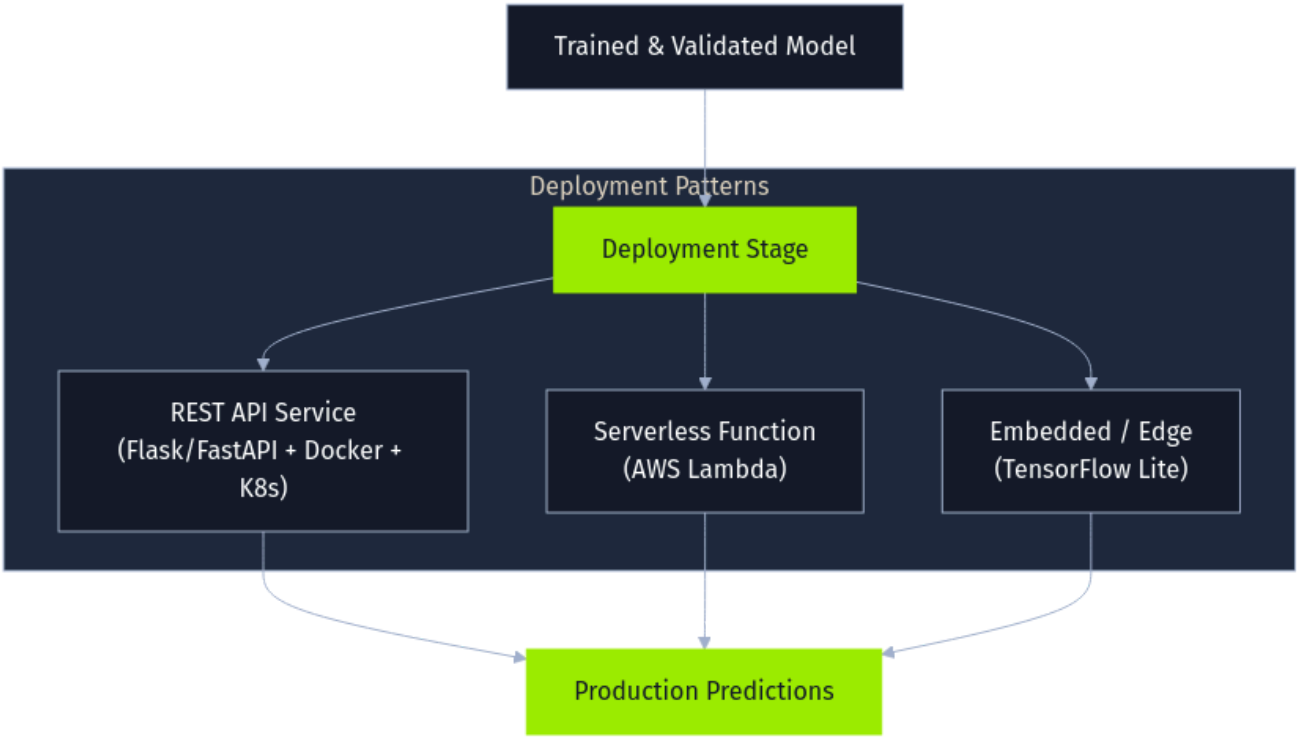
Processed data feeds into the `analysis and modeling` stage. Data scientists explore the dataset in environments like `Jupyter Notebooks`, then train models with frameworks like `PyTorch` or `TensorFlow`. The cycle is highly iterative: pick an algorithm, tune `hyperparameters` with tools like `Optuna`, validate, adjust, and repeat. Cloud platforms bundle much of this lifecycle into managed environments to make it seamless.

From a security perspective, modeling is mostly a downstream consumer. It faithfully learns whatever the data teaches.

If the upstream data was poisoned, the model absorbs the poison. It will not flag it. It will not resist it. The actual attack surface lives in collection, storage, and processing.

Modeling is simply where the consequences become real.

Deployment



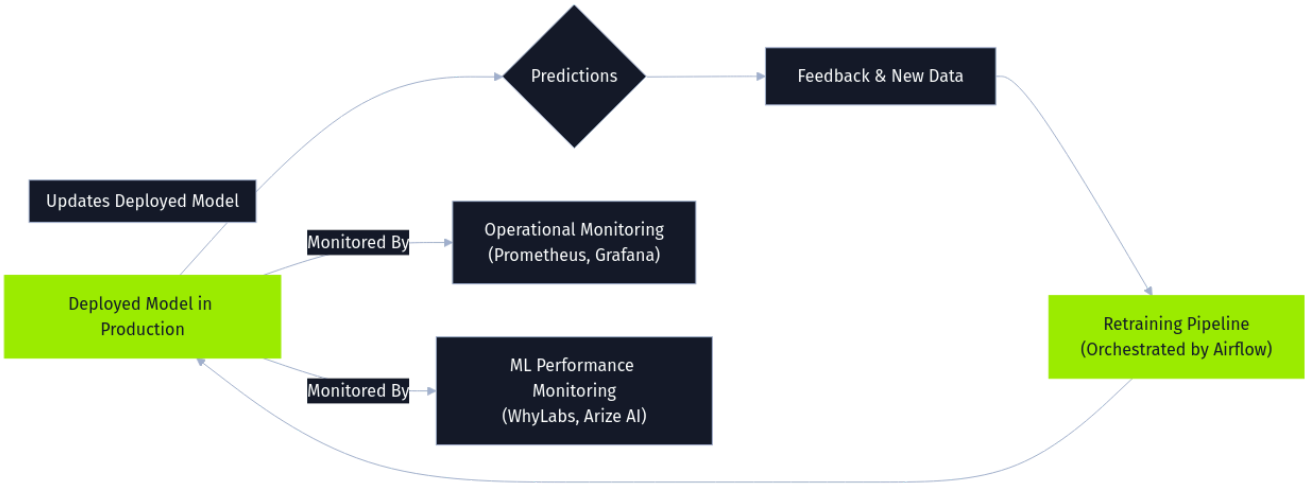
`Deployment` marks the transition from trained artifact to live system. The most common pattern wraps the model in a `REST API` using `FastAPI` or `Flask`, containerizes it with `Docker`, and orchestrates it through `Kubernetes`. Models can also run as serverless functions or be compiled directly for edge devices.

The security concern at this stage is the model file itself.

A model loaded into production is implicitly trusted to produce correct outputs. If that file has been tampered with (whether swapped for a trojaned version or loaded through an insecure deserialization path) the compromise reaches production silently.

Wrong predictions are the minor outcome. Arbitrary code execution is the severe one.

Monitoring and Maintenance



Deployed models do not stay static. `Monitoring and maintenance` tracks both standard operational health (like latency and throughput) and ML-specific metrics like `data drift` and prediction quality.

When performance degrades, the model gets retrained. Feedback from predictions and user interactions is logged, combined with newly collected data, and run through the pipeline again to produce an updated model. Orchestration tools manage these `retraining` cycles automatically.

Here is where the security problem compounds. Retraining is supposed to incorporate new data. That is the whole point. But the pipeline cannot reliably tell the difference between a genuine shift in user behavior and data an attacker deliberately injected.

Malicious data introduced through feedback loops gets absorbed into the next model version. This is `online poisoning` . It is effective precisely because the system is structurally designed to trust its own inputs. The attack exploits the feature, not a bug.

Two Pipeline Examples

The stages above are much easier to reason about with concrete systems. Two examples illustrate how this architecture looks in practice, and exactly where the attack surface sits in each case.

Consider an `e-commerce platform` building a `product recommendation system` . It collects user activity streamed through `Kafka` , along with review text. Raw data lands in a massive data lake on `AWS S3` . `Apache Spark` processes it all: reconstructing user sessions, running sentiment analysis on reviews, and

outputting processed files. Inside `AWS SageMaker`, a recommendation model trains on that data. The model is serialized as a `pickle` file, stored back on `S3`, and deployed through a `Docker`-ized API on `Kubernetes`. Monitoring tracks click-through rates, while user feedback feeding into periodic retraining cycles keeps the model fresh.

Every one of those stages is a potential entry point for an attacker. The stream. The bucket. The jobs. The model file. The loop itself.

A `healthcare provider` building a `predictive diagnostic tool` faces a tighter version of the same fundamental architecture.

Anonymized patient images and clinical notes come from internal hospital systems. Storage is highly restricted and encrypted. `Python` scripts standardize images and extract text features before a heavy `PyTorch` run trains a model on specialized hardware. The validated model deploys via an internal API to a clinical support system. Retraining happens far less frequently and under much stricter controls.

But the fundamental problem persists: incorporating new data still explicitly requires trusting that the incoming data is legitimate.

The stakes are obviously higher because the model's output influences patient care. Ultimately, however, the pipeline's structural vulnerabilities are exactly the same.

Mark Complete & Next